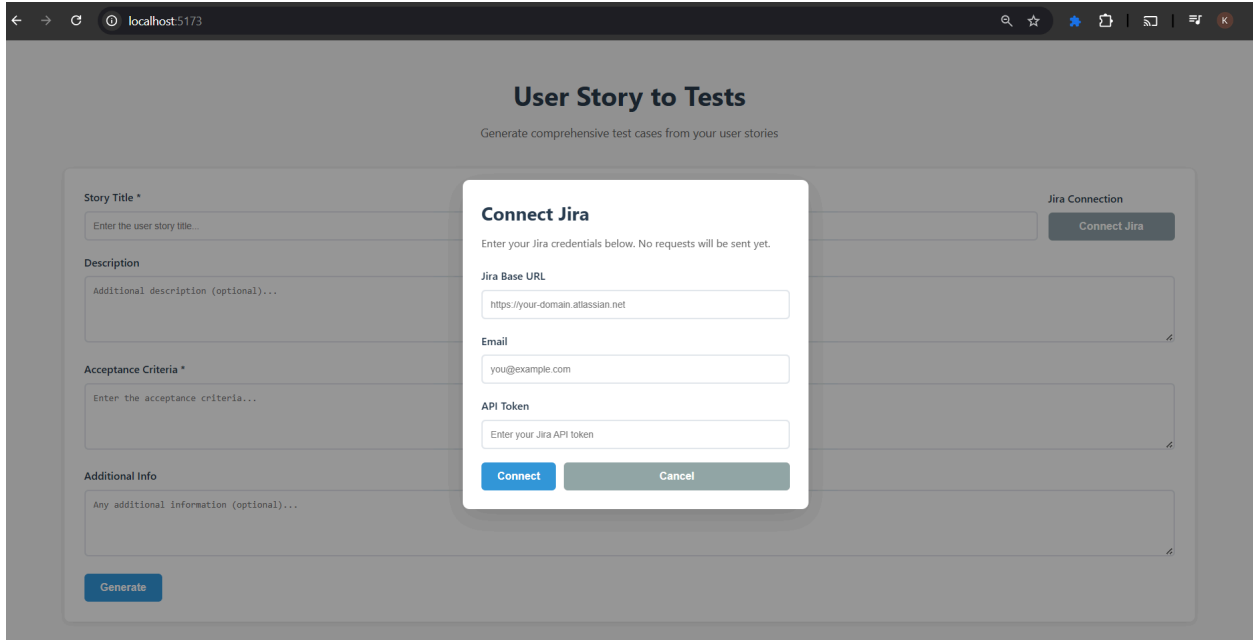


Testcase Generator

Step 1: Added “Connect Jira” button and a dialog box in the UI

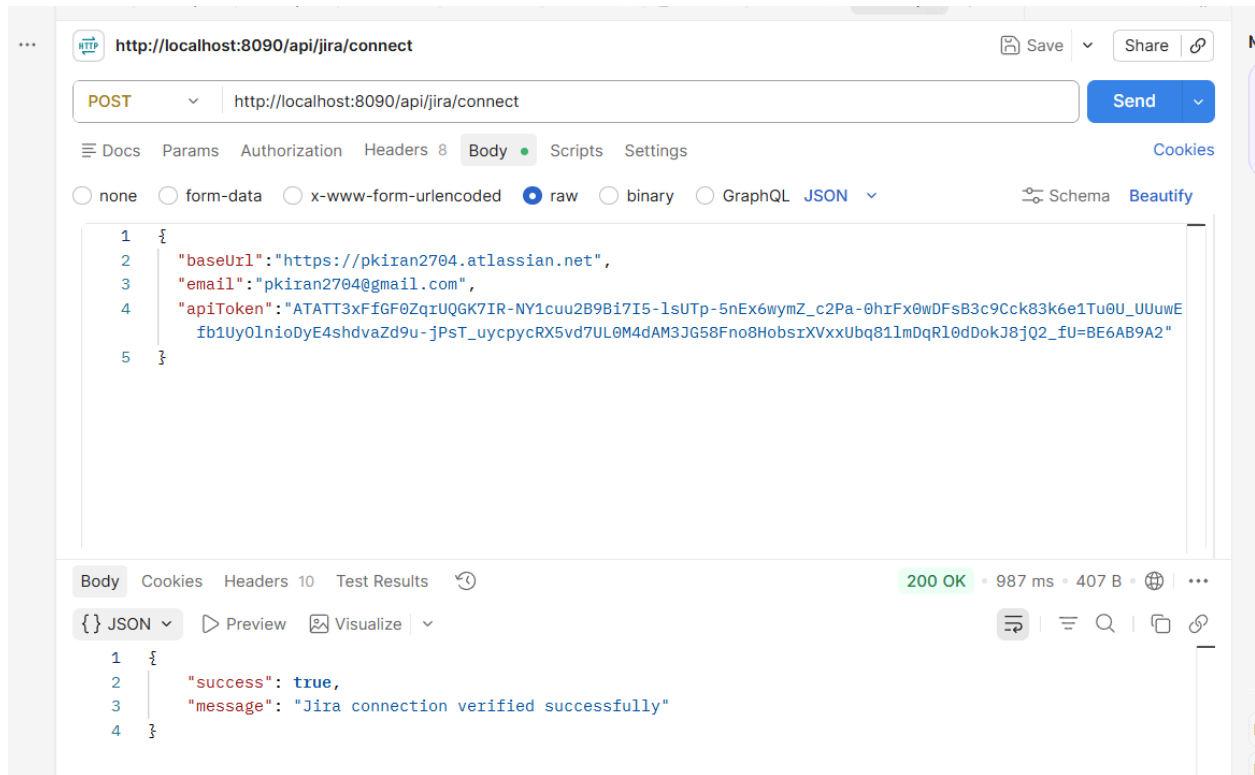


Step 2: Backend API

- Created a new router file backend/src/routes/jira.ts

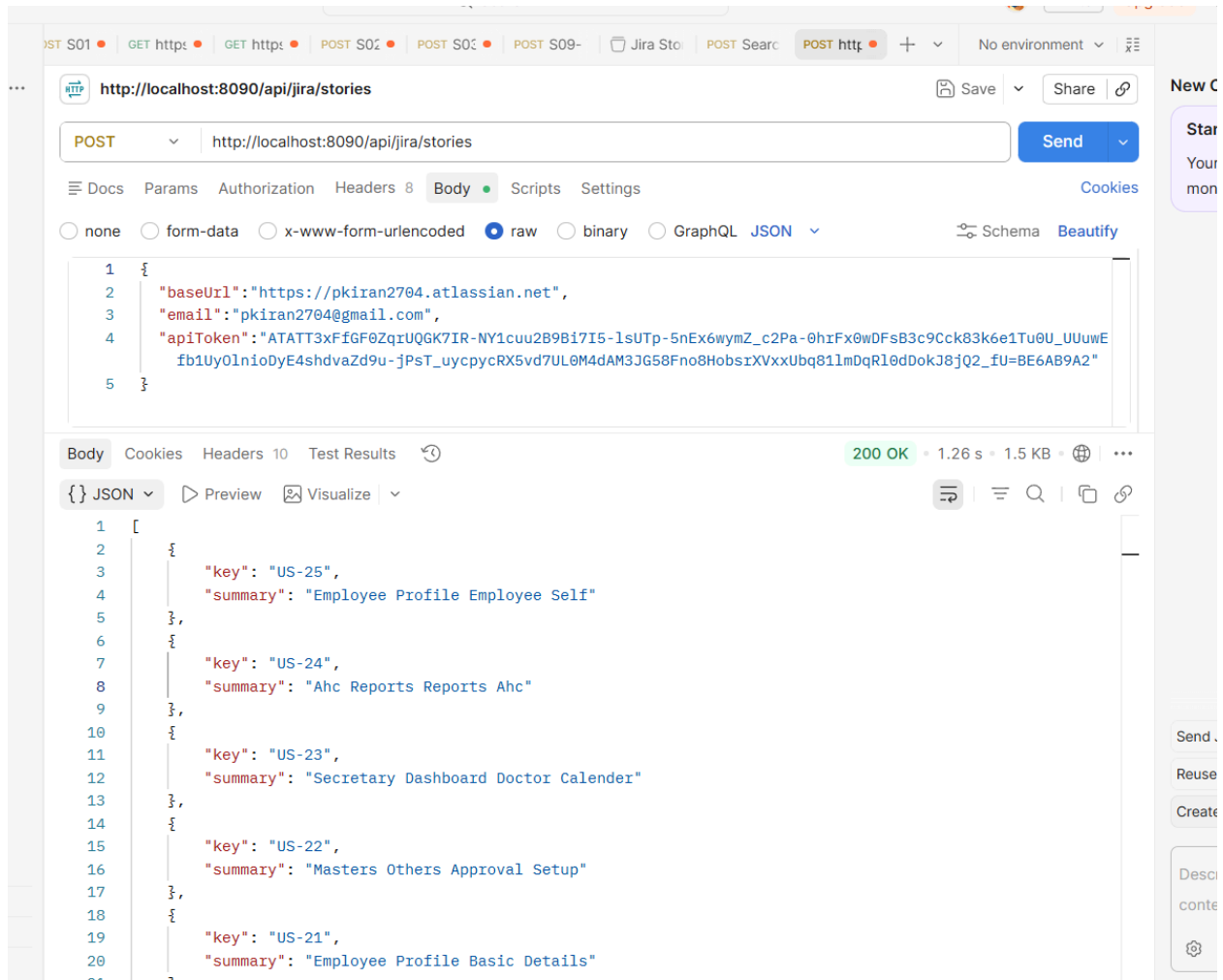
Step 3: Connect to Jira

- Added a new endpoint: POST <http://localhost:8090/api/jira/connect>
- It accepts: baseUrl, email, apiToken
- Authenticate against the Jira REST API
- Return: true/false, message



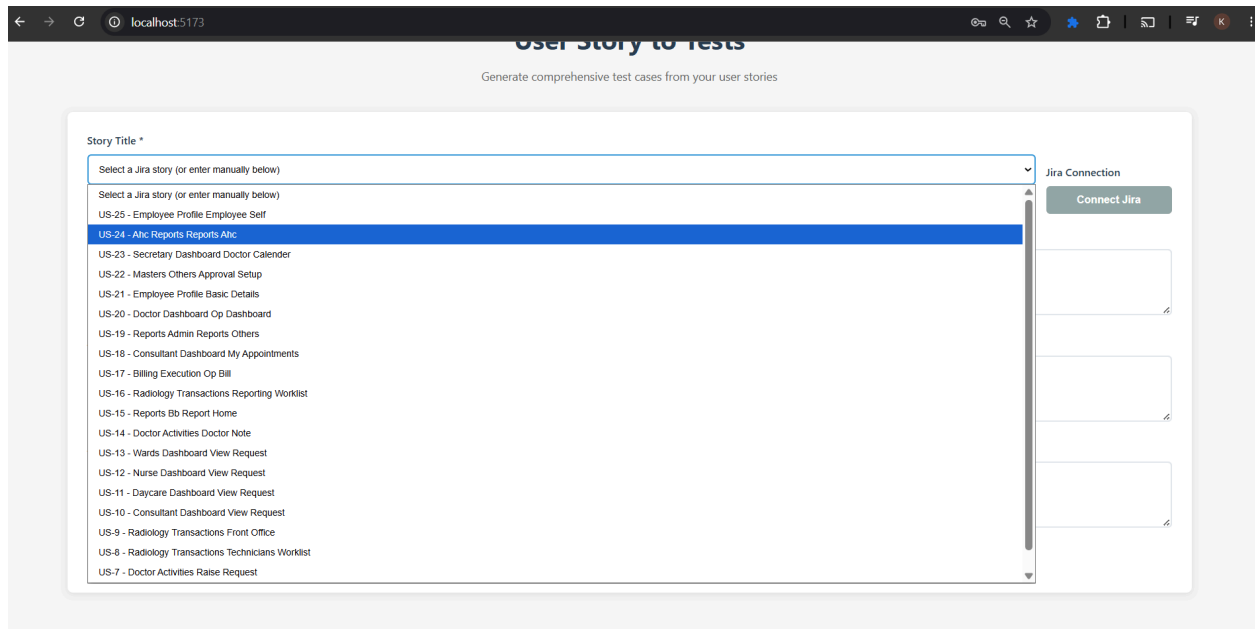
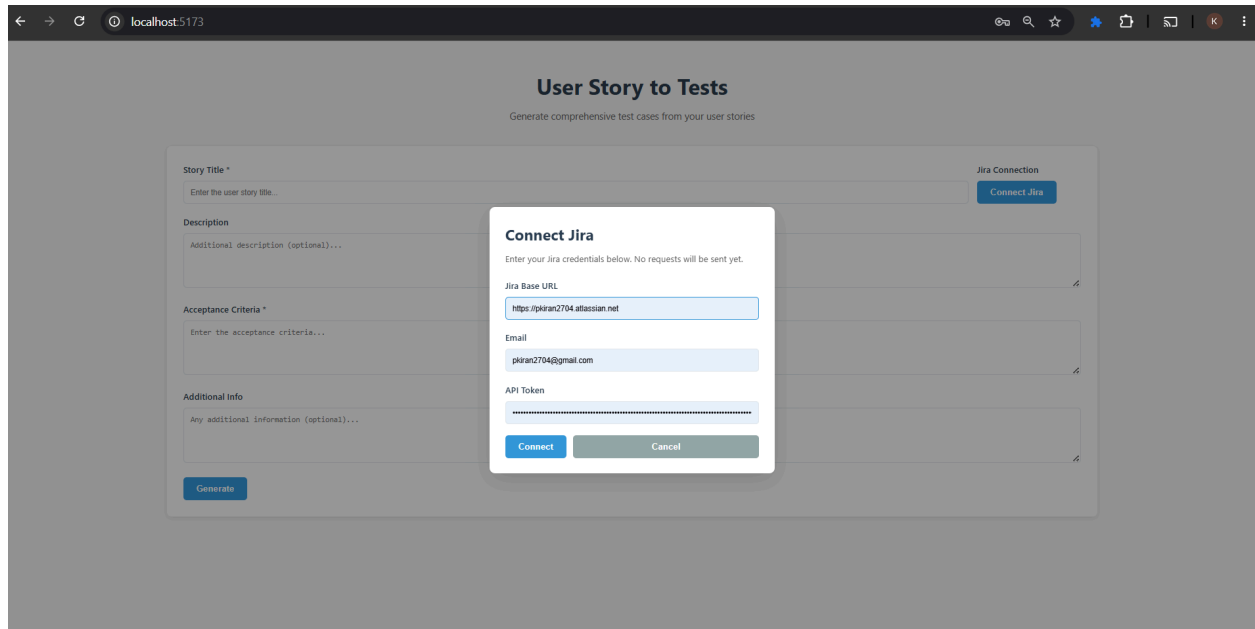
Step 4: Backend: Fetch User Stories

- Added a new endpoint: `POST /api/jira/stories`
- Input: `baseUrl`, `email`, `apiToken`
- Use the Jira Search REST API.
- Fetch the latest Story issues using JQL.
- Return a simplified JSON array containing only: `key`, `summary`
- Reuse the same authentication mechanism already implemented for `/api/jira/connect`.



Step 5: Frontend - Add API and Populate Dropdown

- Added a new function inside the existing frontend API layer that calls: POST `/api/jira/stories`
- Added Dropdown with the list of User Stories



Step 6: Backend Story Details Endpoint & Frontend Wiring

- Added a new endpoint: GET /api/jira/story/:key
- Call the Jira REST API to retrieve the full details for the selected issue.
- Return only the fields required by the current UI: key, summary, description, acceptanceCriteria
- UI calls the backend endpoint to retrieve the full story details.

← → ↺ 🔍 localhost:5173

Generate

Generated Test Cases

30 test case(s) generated • Model: llama-3.3-70b-versatile • Input tokens: 408 • Output tokens: 2614 • Cost: \$0.002306 (est.)

Test Case ID	Title	Category	Expected Result
▶ TC-001	View Report with Existing Data	Positive	The report displays accurate and up-to-date information
▶ TC-002	Generate Report with Existing Data	Positive	The generated report displays accurate and up-to-date information
▶ TC-003	View Report with No Data	Negative	The report displays a message indicating no data is available
▶ TC-004	Generate Report with No Data	Negative	The generated report displays a message indicating no data is available
▶ TC-005	View Report with Invalid Credentials	Authorization	The system displays an error message indicating invalid credentials
▶ TC-006	Generate Report with Invalid Credentials	Authorization	The system displays an error message indicating invalid credentials
▶ TC-007	View Report with Large Dataset	Non-Functional	The report displays accurate and up-to-date information without performance issues
▶ TC-008	Generate Report with Large Dataset	Non-Functional	The generated report displays accurate and up-to-date information without performance issues
▶ TC-009	View Report with Special Characters	Edge	The report displays accurate and up-to-date information without issues
▶ TC-010	Generate Report with Special Characters	Edge	The generated report displays accurate and up-to-date information without issues
▶ TC-011	View Report with Empty Fields	Negative	The report displays a message indicating empty fields

Step 7: Added “Generate Feature File” button in UI

- Added a new button for Feature File Generation
- Modified Frontend APIs to have 2 values for the generationType parameter: testcase , feature
- Wired the buttons with the respective parameter values

The screenshot shows a web browser at localhost:5173 displaying a form titled "User Story to Tests" with the subtitle "Generate comprehensive test cases from your user stories". The form contains four text input fields: "Story Title *" (placeholder: "Enter the user story title..."), "Description" (placeholder: "Additional description (optional)..."), "Acceptance Criteria *" (placeholder: "Enter the acceptance criteria..."), and "Additional Info" (placeholder: "Any additional information (optional)..."). To the right of the "Story Title" field is a "Jira Connection" section with a "Connect Jira" button. At the bottom of the form are two buttons: "Generate Test Cases" and "Generate Feature File".

Step 8: Update Backend

- Made changes to accept new generationType parameter
- Add support for a second prompt when generationType = "feature"
- The feature prompt instruct the LLM to generate a valid Gherkin Feature File
- And updated the [generate.ts](#) to switch the prompts based on user input

This screenshot shows the same web application after processing. The input fields are populated with example data: "Story Title" is "As a user, I want to doctor dashboard op dashboard so that the related functionality works as expected.", "Acceptance Criteria" is "Given a user with valid access, when they open the dashboard, then relevant data and visualizations should be displayed correctly without delay.", and "Additional Info" is "Any additional information (optional)...". The "Generate Test Cases" and "Generate Feature File" buttons are still present. Below the form, a new section titled "Generated Feature File" displays the output in a code block:

```
Feature: US-20 - Doctor Dashboard Op Dashboard
Scenario: Valid user accesses the dashboard
  Given a user with valid access
  When they open the dashboard
  Then relevant data and visualizations should be displayed correctly
  And the data and visualizations should be displayed without delay
```